Introducing Ptables

Taking One More Crack At IPTables



Jamal Hadi Salim & Nabil Bitar

What is Policy Tables (Ptables)?

Minimalist alternative to the trio of IPtables + IPset + Conntracking

Current implementation: eBPF Kernel code with userspace control cli

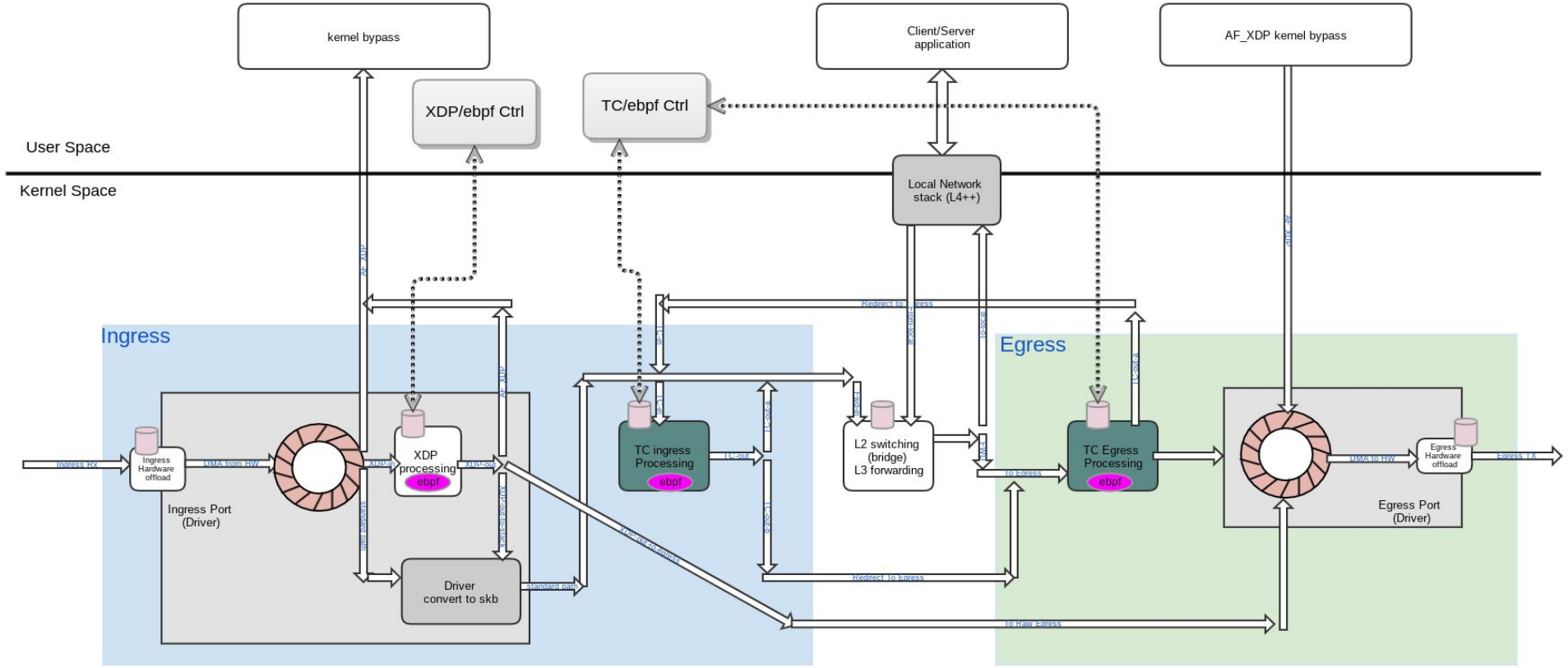
Motivation: $$$$

- Performance, Performance, Performance
  - CAPEX of buying more/bigger servers
  - OPEX for more/bigger servers
  - PPS normalized over CPU utilization
  - PPS normalized over Latency
    - What is the cost of being 1 microsec worse?



Design Approach And Goals

Ptables Current Attachment Points



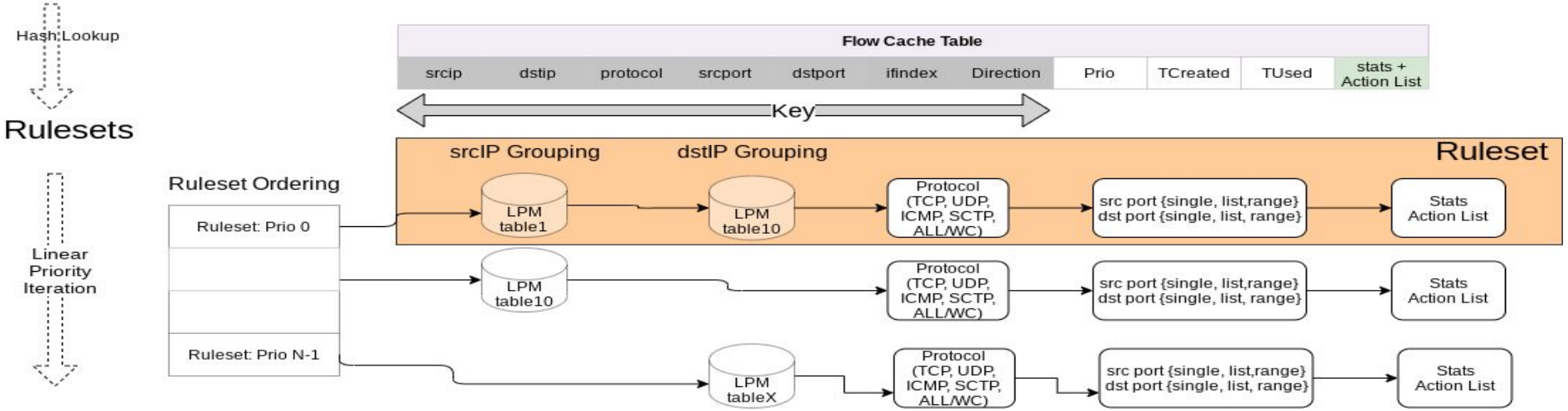
Policy tables(Ptables): Design Goals vs IPtables

1. Performance (See our other talk)
    - higher throughput, lower latency, improved control path
2. Minimalism in Flexibility
    - to infuse just enough functionality for the required task
3. Extensibility
    - Innovate beyond the trio's semantics when needed
4. Usability Metric
    - IPTables + IPSet +conntrack level of complexity

Ptables: General abstraction

Lookup Order:

1) *Optional* Flow Cache Table (exact match) (equivalent to Iptables *conntrack*)
2) If miss on FCT then search Ruleset Tables (equivalent to *Iptables* rules IP address grouping based on *IPSet*)
    a) On match update FCT
    b) On miss: Default rule



Policy Interface Goals

1. Support ingress and egress policy enforcement on a netdevice group (one or more devices)
2. Match on 5 tuples (can be extended):
    - src/dst IP CIDR groups, IP protocols (TCP, UDP, SCTP, ICMP), src/dst ports (or icmp code/types)
    - Tuples in a ruleset could be absent
3. Support transport port grouping: ranges and lists
4. Support explicit priorities
5. Support statefulness/conntracking/caching
    - Basic on per-port 5 tuples exact match
6. Per-Flow details (via cache)
    - Accounting statistics
7. Semantically equivalent to IPTables + IPSet + conntrack

Performance Goals

Higher throughput normalized over CPU and Latency

How?

- Our initial implementation is based on eBPF but is not restricted to it
- We will use all the necessary infrastructure to achieve our goals
  - We plan to go beyond ebpf in the future (offload etc.)

Extensibility Goals

- Abstraction in the policy interface should work the same way for both S/W and H/W dataplanes
  - Any complexities in transformation for different target hardware should not affect the user interface

Flexibility Goals

Compile only what you need

Got this today but: I should get this or more if i want





Usability Goals

Initial Goal:

> The Trio craftsperson should be able to use the interfaces

Someone Has Tried To Implement this with eBPF?

Yes! but all trying to be evolutionary as IPtables management clones

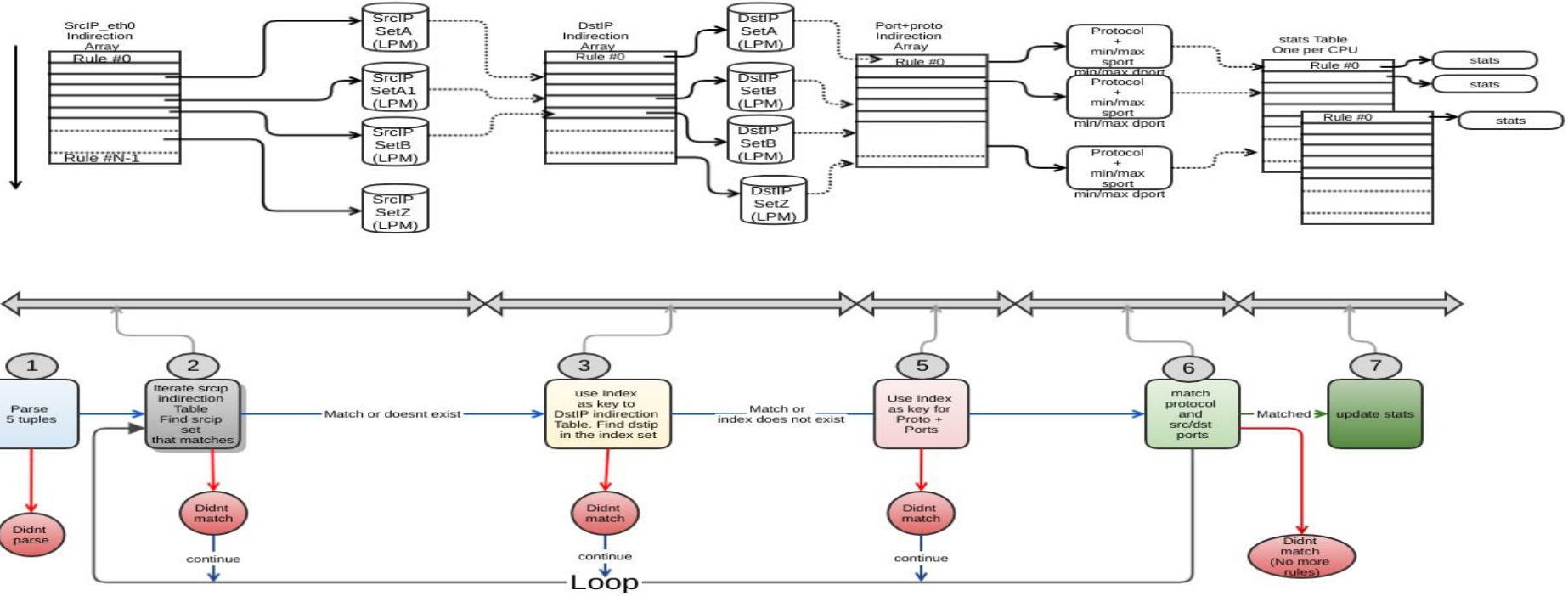
- Fulvio Riso et al: shim libiptc to transform calls into ebpf
- Bpfilter: Shim into kernel level system call levels and transform them to ebpf backend
- Harald Welte's suggestion: IPtables cli option to invoke ebpf backend

Ptables instead adds an *r* to *evolution*

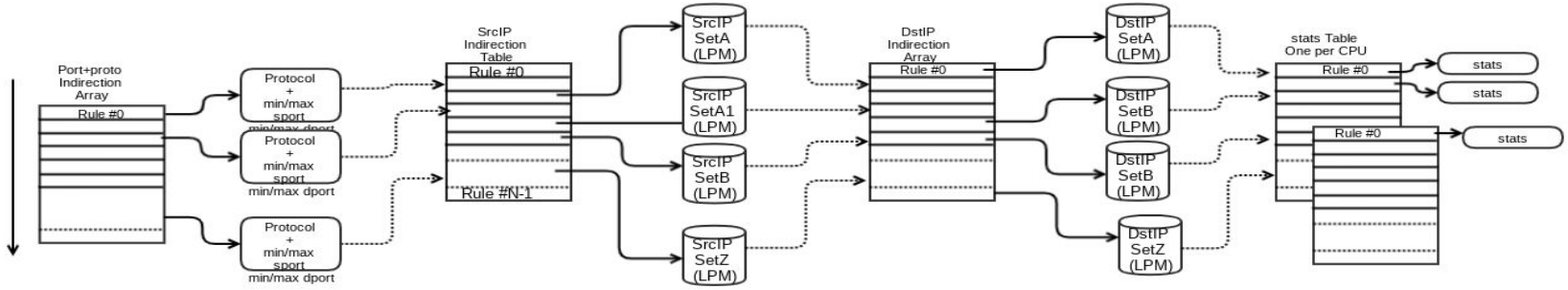
Ptables Dataplane Implementation: Current State

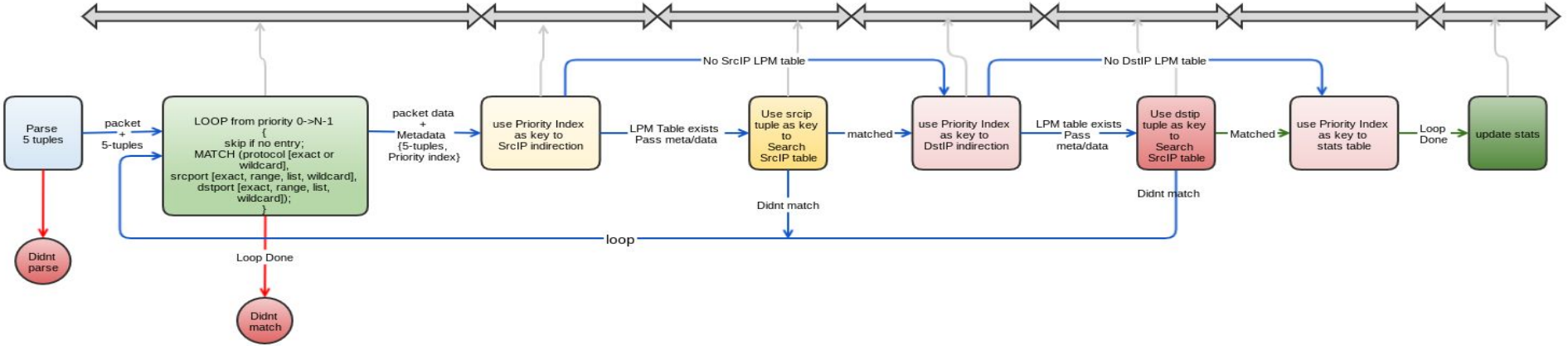
- eBPF
- Based on RHEL 8.3
  - Do not use newer kernel release features not backported to RHEL 8.3
- Based on a common control interface for both XDP and TC eBPF
- Using clang 11

Ptables Initial eBPF Design



Eventual eBPF Design





Semantics: CIDR Groups Examples

```
$IPSET_BIN create itable1 hash:net maxelem 10 counters

$IPSET_BIN add itable1 4.172.179.63/18

$IPSET_BIN add itable1 64.31.7.15/23

$IPSET_BIN add itable1 89.44.237.230/22

$IPSET_BIN add itable1 56.49.103.36/16

$IPSET_BIN add itable1 31.14.209.46/20

$IPSET_BIN add itable1 119.63.247.140/24

$IPSET_BIN add itable1 124.195.82.112/32
```

```
$PTABLES_BIN create table itable1

$PTABLES_BIN add table itable1 18.146.2.60/22

$PTABLES_BIN add table itable1 123.97.65.34/19

$PTABLES_BIN add table itable1 28.55.38.251/24

$PTABLES_BIN add table itable1 66.110.72.187/28

$PTABLES_BIN add table itable1 25.51.65.151/12

$PTABLES_BIN add table itable1 12.129.102.206/16

$PTABLES_BIN add table itable1 102.81.79.68/24
```

Semantics: Ruleset Creation/Binding Examples

$IPTABLES_BIN -A -i $DEV INPUT -m set --match-set itable9 src -m set --match-set itable6 dst -p udp -m multiport --sports 20000:25000 -m multiport --dports 8080 -j ACCEPT

→ $PTABLES_BIN create rule dir ingress dev $DEV prio 0 src itable9 dst itable6 protocol udp multiport sprange 20000:25000 dport 8080 action accept

$IPTABLES_BIN -A -i $DEV INPUT -m set --match-set itable7 src -m set --match-set itable27 dst -p udp -m multiport --sports 433 -m multiport --dports 88 -j ACCEPT

→ $PTABLES_BIN create rule dir ingress dev $DEV prio 1 src itable7 dst itable27 protocol udp multiport sport 433 dport 88 action accept

$IPTABLES_BIN -A -i $DEV INPUT -m set --match-set itable4 src -m set --match-set itable5 dst -p udp -m multiport --sports 2,25 -m multiport --dports 8080 -j ACCEPT

→ $PTABLES_BIN create rule dir ingress dev $DEV prio 2 src itable4 dst itable5 protocol udp multiport splist 2,25 dport 8080 action accept

Ptables Dataplane Implementation: Current State

- eBPF
- Based on RHEL 8.3
  - Do not use newer kernel release features not backported to RHEL 8.3
- Based on a common control interface for both XDP and TC eBPF
- Using clang 11

eBPF - Challenges and Implementation

Teething: Not Ready For The Faint Hearted

- *Tooling, tooling, tooling*
  - eBPF still evolving: *moving fast*
  - Keeping up requires guessing sometimes: Is it the kernel? Compiler? Verifier?
- Kernel changes
  - Latest Example: BTF introduction in the kernel implies changing pahole
    - This is probably the most intrusive kernel related issue I have ever seen
  - Compiling kernel dirs for test and samples could be challenging depending on what version of clang or other tools you have installed
    - Also tainted by all the clever reusability trickery (instead of simple samples)
- iproute2
- libbpf: Best of the lot
- Verifier challenges: More on next few slides
- Compiler: More next slide

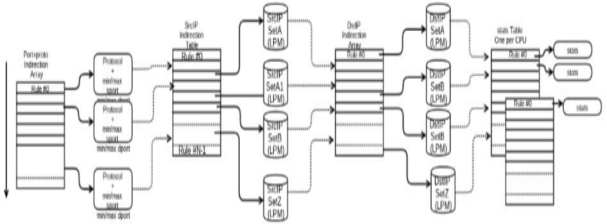
Challenges: Clang

- We never trusted the compiler in the earlier days
- Ran into bugs in different versions of the compiler
  - Most recent bug:
    - Using zero-initialized variable as a key generates bogus opcodes (rejected by the verifier)
- Keeping up with compiler changes implies you need to build from git
  - Building Clang has vastly improved over the last six months
    - For a lifesaver see: https://apt.llvm.org
- Sometimes incompatible to have user code built using gcc and kernel code using clang

Challenges: Verifier

- 4K Instructions in earlier kernel tested - caused challenges
- Upgraded to later Kernel that supports 1M instructions (supported in RHEL 8.3)
- Using bpf-to-bpf calls is usually a burden for verifier messages
  - Forced inlining of the whole program
    - - mllvm --inline-threshold=10000000

Challenges: Loops



- We need loops to iterate through Rulesets
- Loops missing in earlier tested kernel
  - Initial approach was to use tc priorities - one per ruleset
  - Workaround with clang #pragma to unroll loop
    - Hit loop size limitation with # of instructions
  - 5.3 onwards introduce bounded loops
    - Increases complexity metric for the verifier (more on this later)

Challenges: Code Complexity

- Turns out it is not just about number of instructions!
- A *Prog* with 3200 instructions rejected by verifier because of branches
  - Introducing branches inside for-loops susceptible to path explosion
    - Only way to find out is when you try to load the program...
    - Then you get unhelpful messages from the verifier
      - They often point to areas of code that have nothing to do with issue
  - Essentially poor development productivity for complex programs

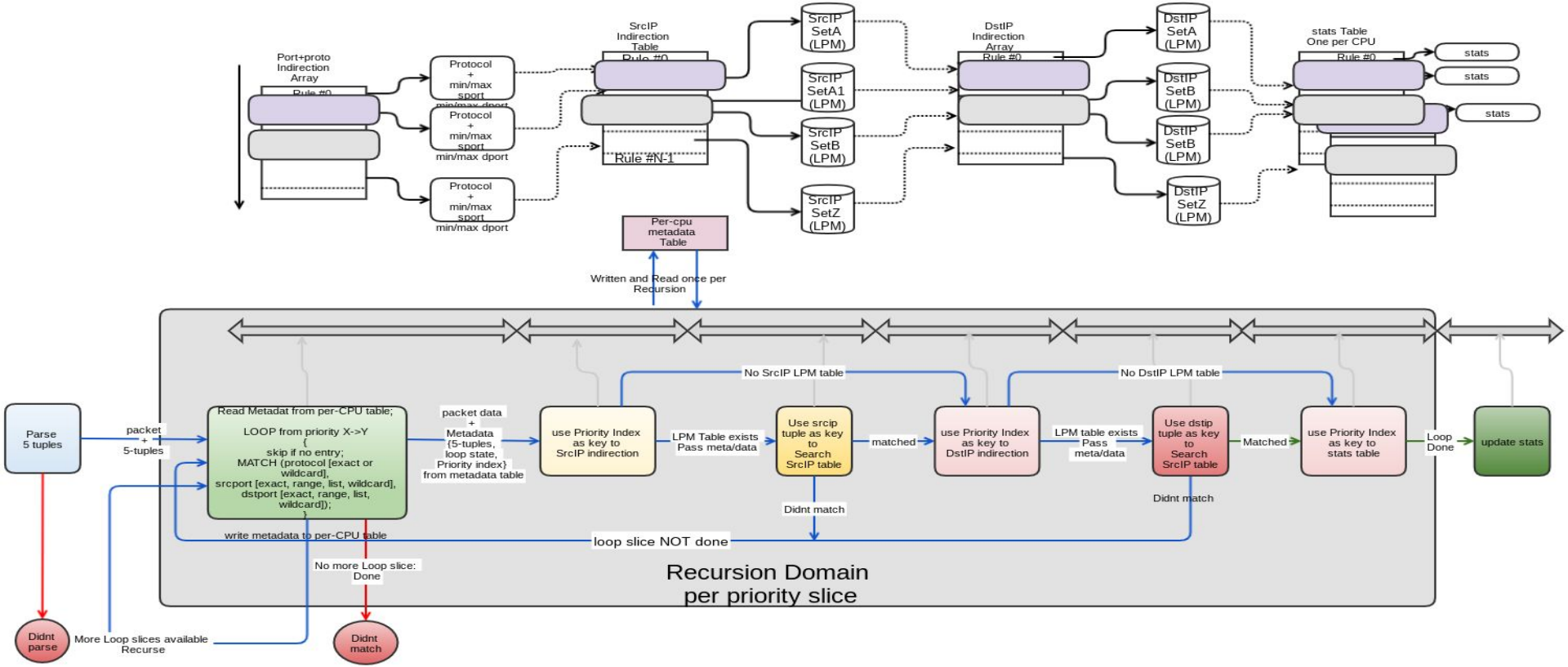
Overcoming Challenges: Loops And Code Complexity

- Initial implementation on TC/eBPF used array of programs
  - Bounded loop inside each program
  - Carry loop and flow state in metadatum skb->cb (20B).
  - Worked great
    - Could do very large loops (a few thousand, bounded by tail call limit) that can map up to 1M instructions per program
    - Can have up to 32 programs → supporting up to 32 M instructions in total
    - Then came the need to support XDP...

Challenges: Same Code Base for XDP + TC

- Need to support both TC and XDP with the same policies
- Adding XDP support resulted in face-palm
  - No ability to use skb->cb equivalent with XDP
  - Obvious answer: For XDP use the context metadata area
    - Performance went down >= 10%
- Back to the drawing board
  - Use per CPU metadata table
    - Idea borrowed from Fulvio Risso et al
      - Works well because we run to completion
  - Means multiple read/write to table per-packet but performance improved relative to accessing XDP metadata

Implementation: Turns Out We Can Recurse….



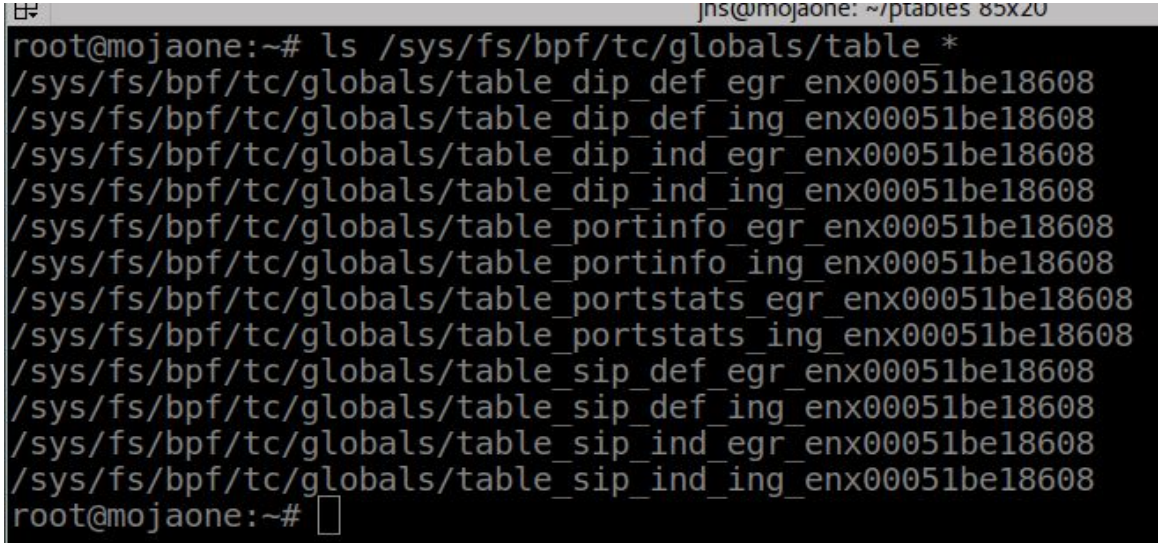
Challenges: Global Tables



- Desire Per-port-per-direction tables
  - Tables in eBPF are global
  - Iproute2::tc interfacing got in the way
- Introduce compile time naming
  - Would have been nice to be able to rename existing tables
  - Hope the new libbpf tc API works, we could create tables with correct names
    - Works with libbpf XDP tables

Challenges: Maps of Maps

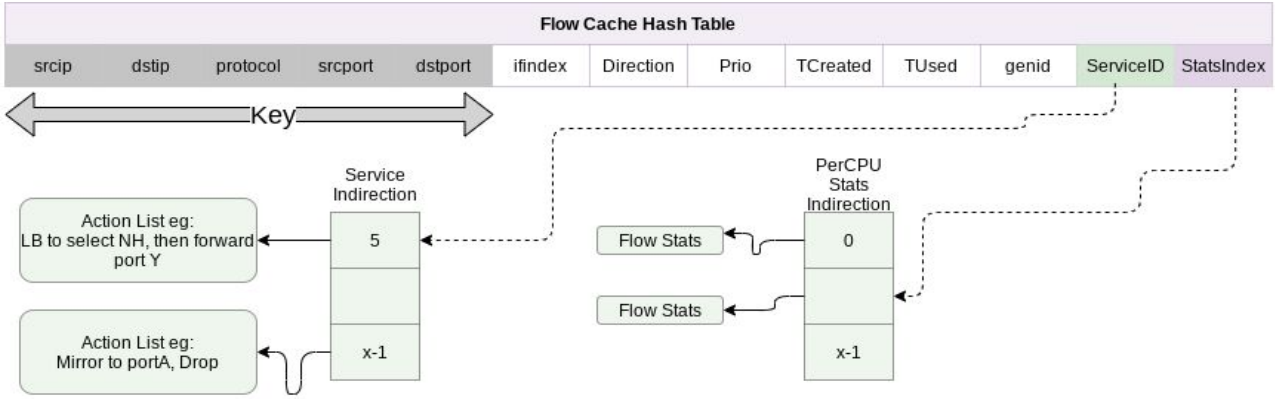
- Cannot have heterogeneous maps of maps
- Cannot have pointed-to-maps (of same type) to be of different sizes
- Cannot have an empty map-of-map i.e. one without at least one entry



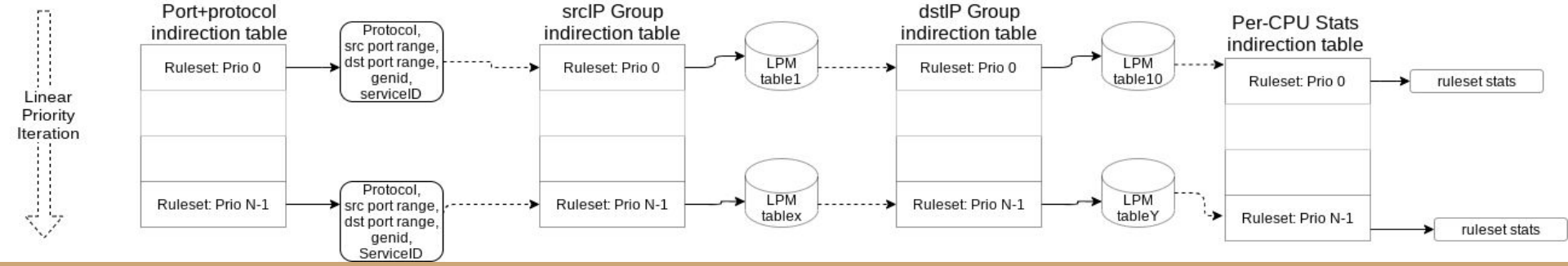
Performance Challenges: Hash vs Array

- Hash cheaper in memory when not fully utilized
  - Arrays pre-allocate
  - RAM is a lesser issue for us (we have > 100G of RAM!)
- Performance was 20-30% cheaper on Array vs hash depending on the map size
  - We fixed our indirection tables to be arrays
  - Lesson: Clever optimization can bite you..

Introduce Caching



Rulesets



Challenges: Caching

- Most basic implementation is eBPF LRU
- Challenges with LRU
  - Does not meet our requirements to collect per-flow accounting
    - Eviction removes cache entries without consulting us
  - Thrashing if not properly commissioned
  - Can't pin flows so that they are not subject to eviction
  - Tricky if we update a ruleset which may have flows previously cached

Overcoming Challenges: Caching

- Invented our own caching using hash maps
  - Can pin flows (to not be evicted)
  - Flow accounting works well
    - We control eviction
  - Eviction on ruleset update is at the moment brute-force flush-the-cache
  - How to deal with cache full?
    - Global per-port-direction boolean flag *cacheisfull* is set
      - Event sent to user space to notify
      - Don’t create new entries when that flag is set
      - User space will evict an entry and unset the flag
        - New cache entries can be now added

Challenges: XDP Limitations

- Large MTU
- LRO
- Can’t broadcast
- Timestamps
- Etc
- See XDP workshop discussions

Potential Future Work

- Open Source
- Write tool to convert from iptables-save to Ptables
- Support for fragmentation
- Bidirectional state
- IPv6 support
- Support for other OSes: Windows, The BSDs
- Rust based control plane

Acknowledgements

- Paul Mininni
- Pedro Tammela

References

- https://netdevconf.info/0x15/session.html?talk-linux-ACL-performance-analysis
- https://netdevconf.info/0x12/session.html?toward-an-ebpf-based-clone-of-iptables
- https://lwn.net/Articles/822744/
- https://lore.kernel.org/bpf/c72bac57-84a0-ac4c-8bd8-08758715118e@fb.com/T/

Images

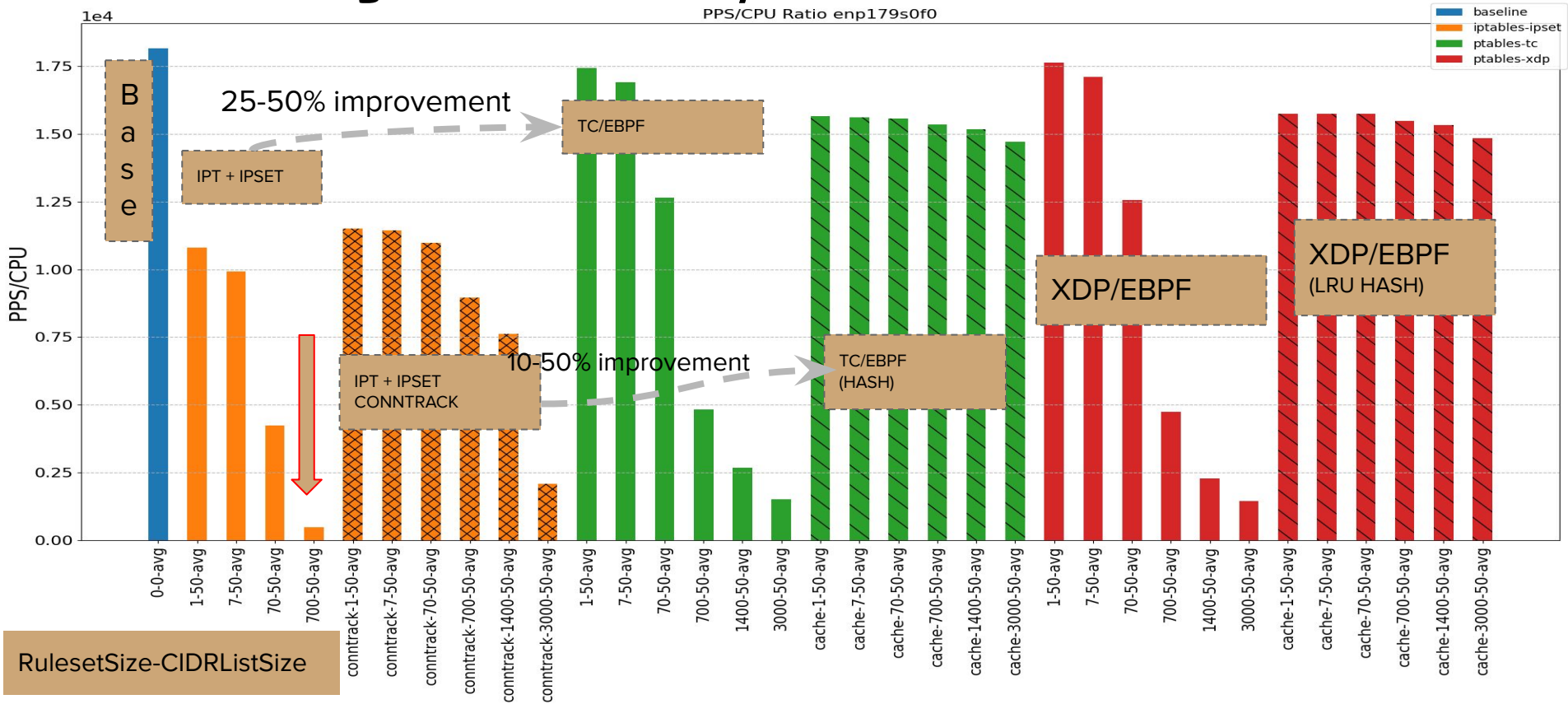
- Berlin Wall coming down
  - https://www.smithsonianmag.com/history/amazing-rare-photographs-berlin-wall-coming-down-180953278/
- Grace Hopper
  - https://images.app.goo.gl/3UQqaNfJmrmZdiJv7
- Big Swiss Army knife
  - https://uncrate.com/giant-swiss-army-knife/
- Tiny knife:
  - https://images.app.goo.gl/6qQbMmBCzyFNmEMU6

Q &A

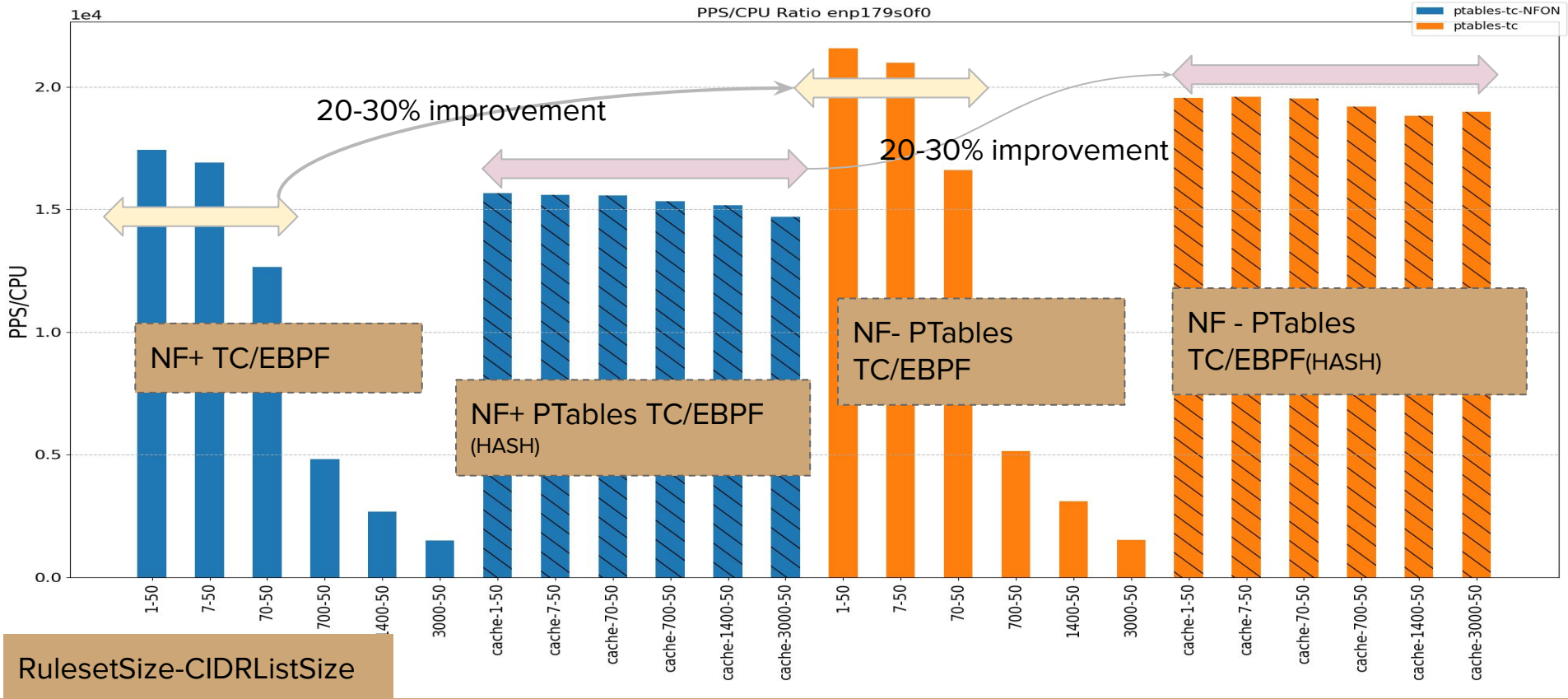
Backup Slides

Datapath Performance Comparison: The Trio vs PTables

Forwarding: NF ON PPS/CPU (Higher Better)

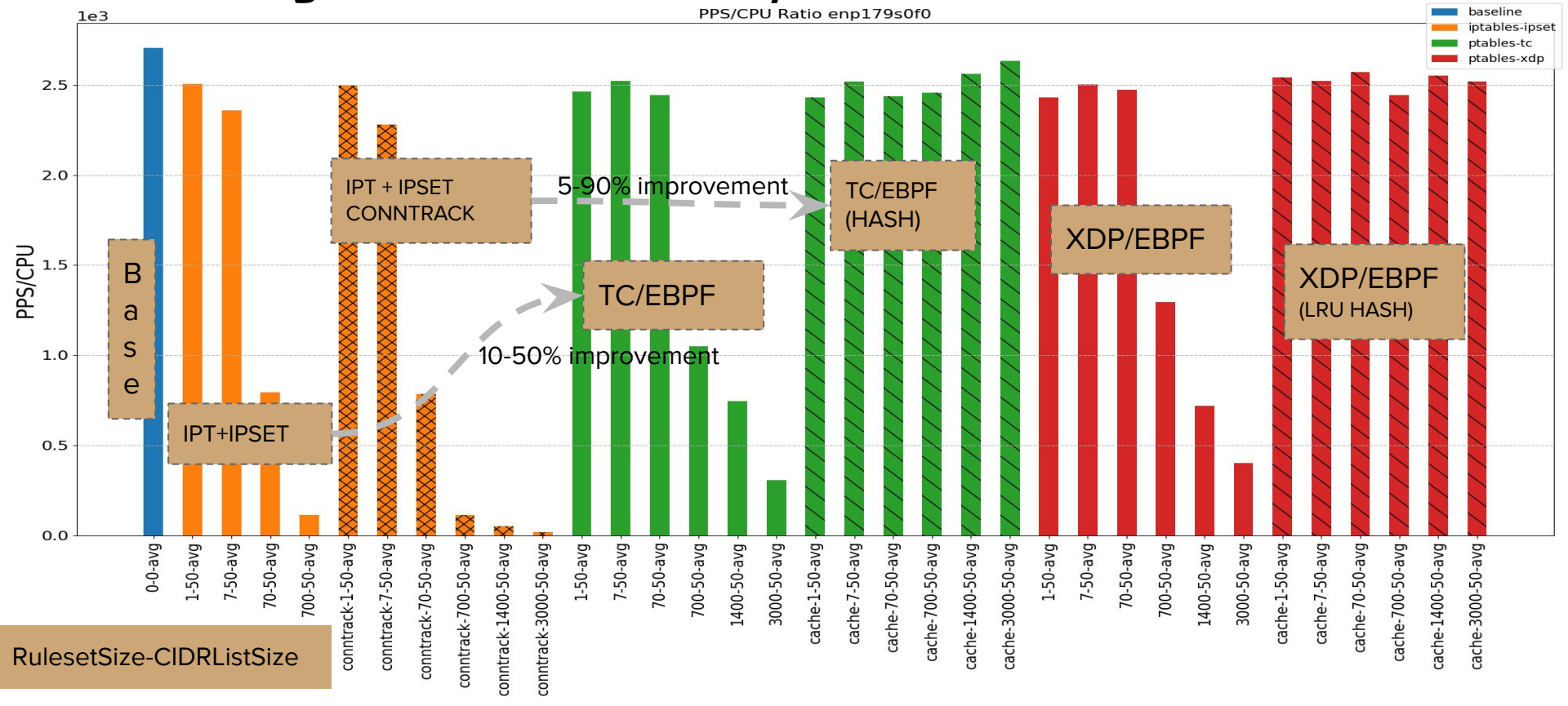


Forwarding TC NF OFF vs NF ON PPS/CPU

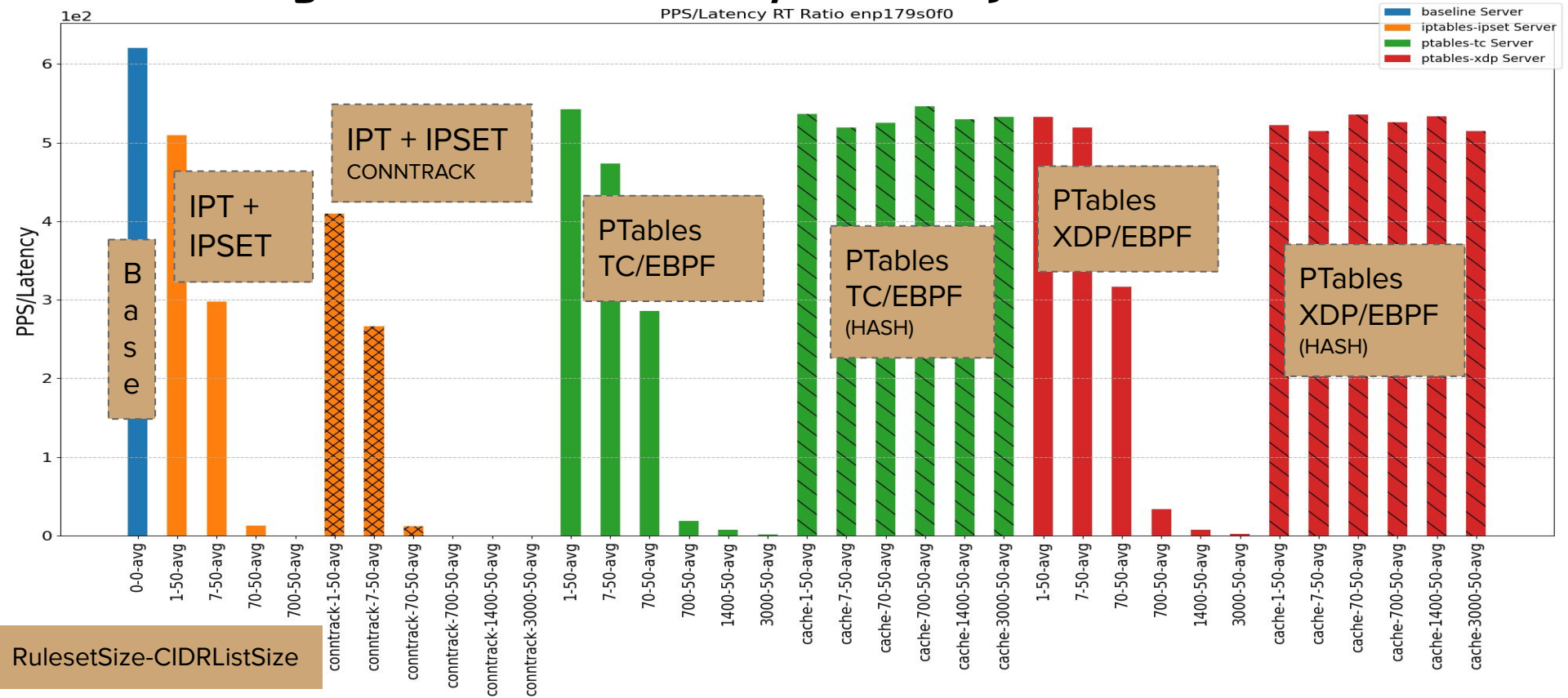


RulesetSize-CIDRListSize

Host: Ingress NF ON PPS/CPU (Higher Better)

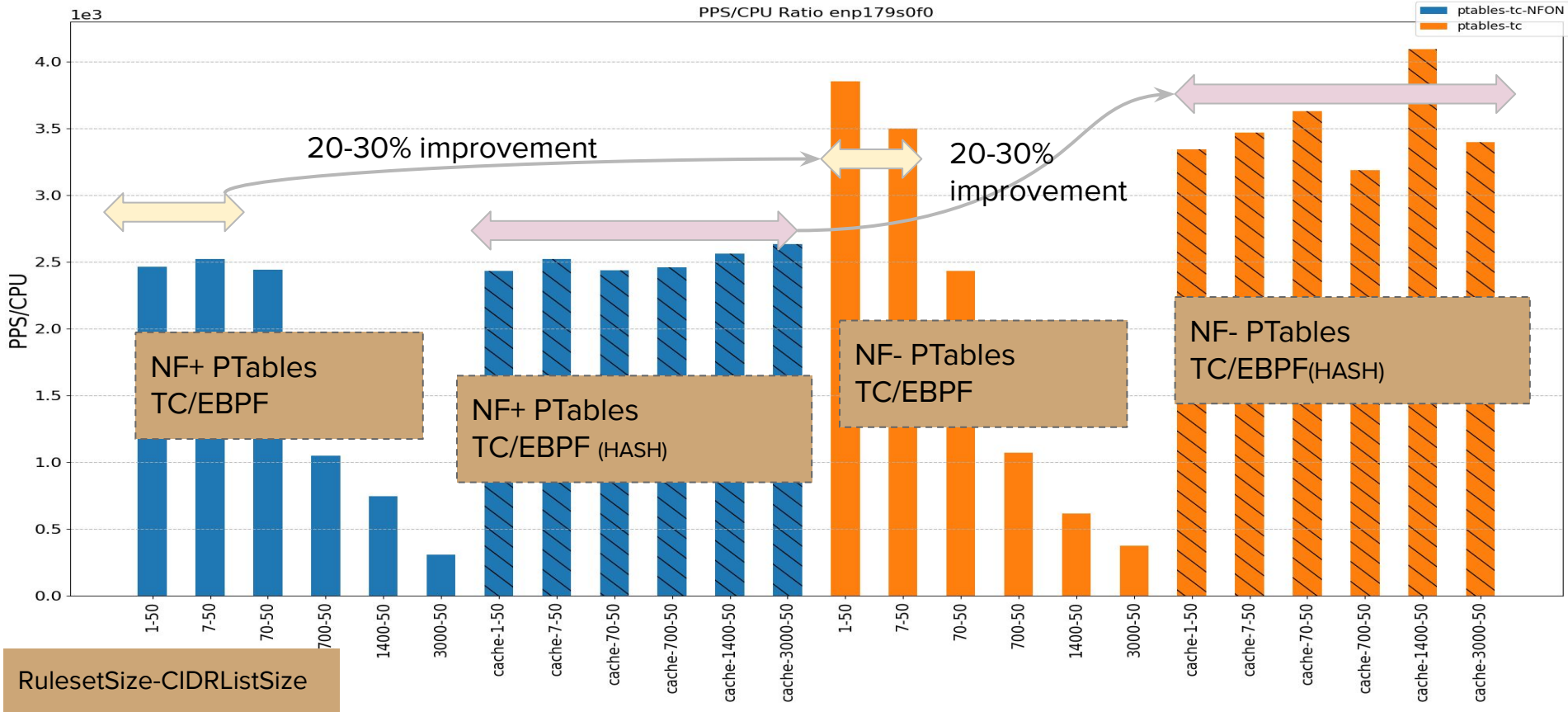


Host: Ingress NF ON PPS/Latency (Higher Better)

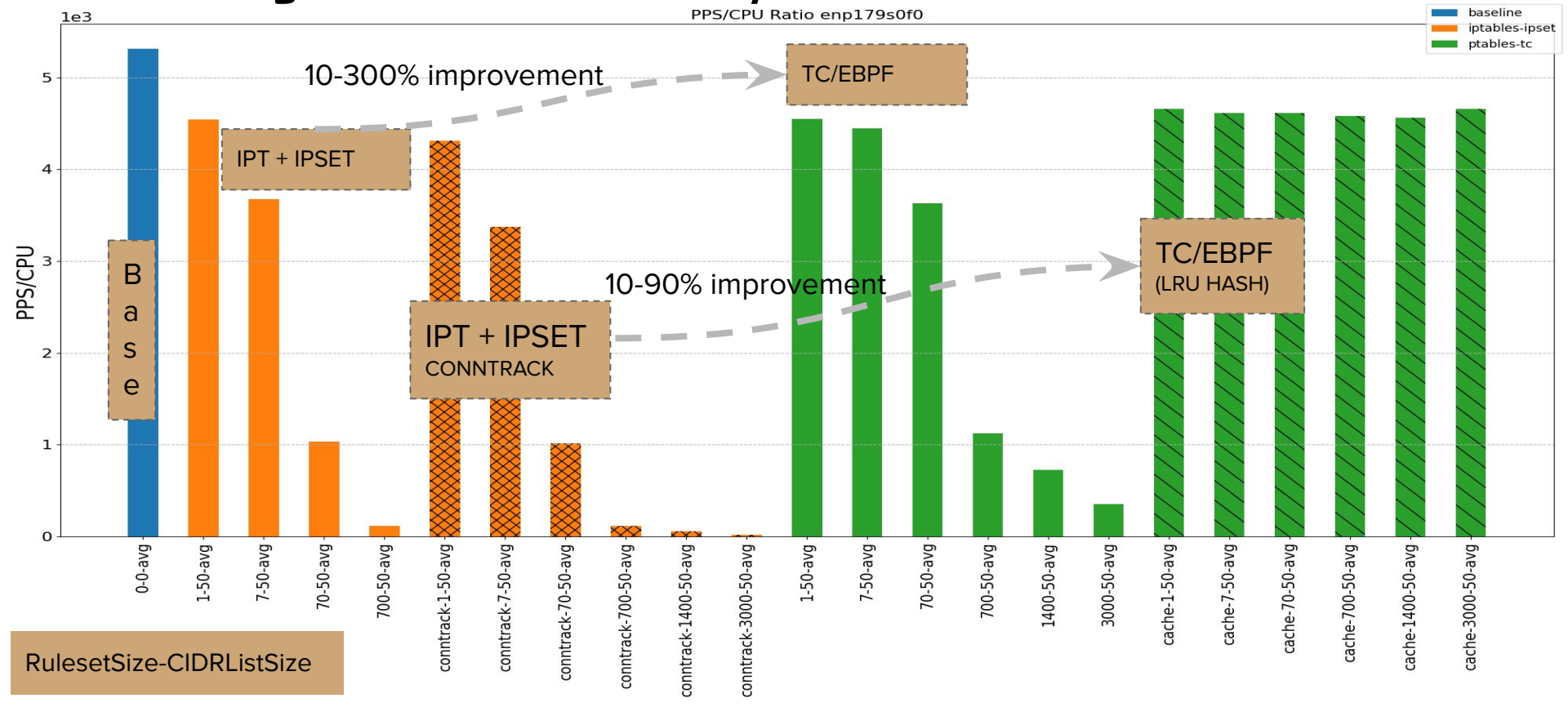


RulesetSize-CIDRListSize

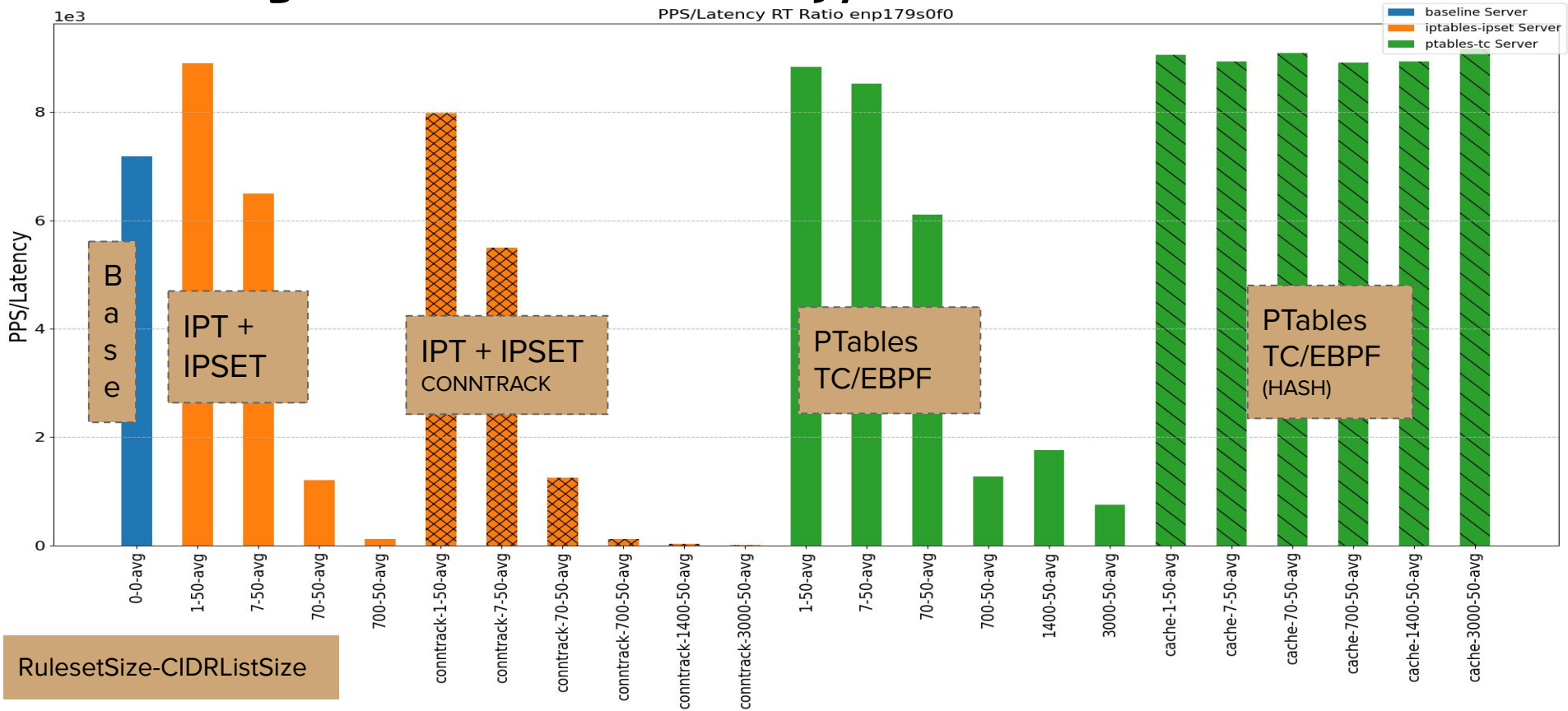
Host Ingress TC NF OFF vs NF ON PPS/CPU



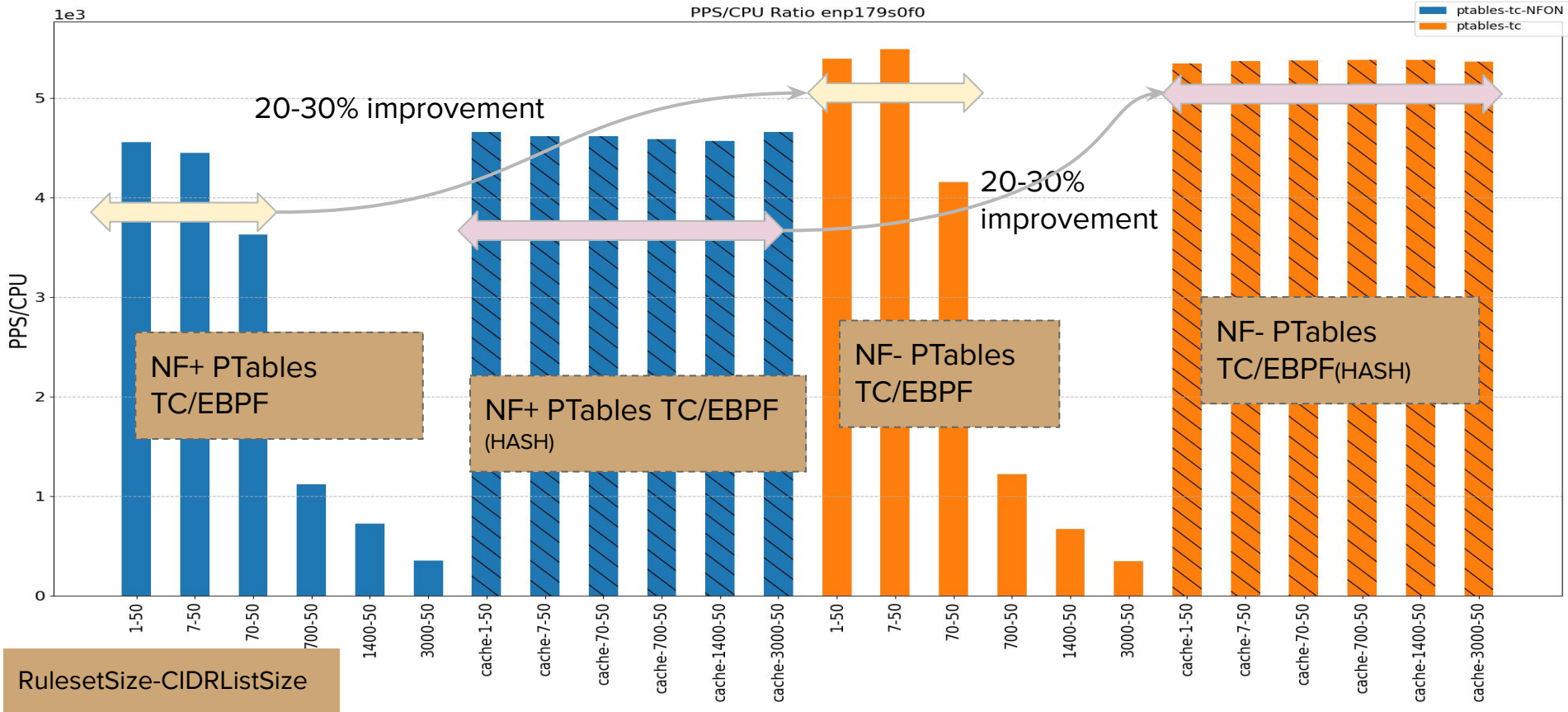
Host: Egress NF ON PPS/CPU (Higher Better)



Host: Egress NF ON Latency/CPU (Higher Better)

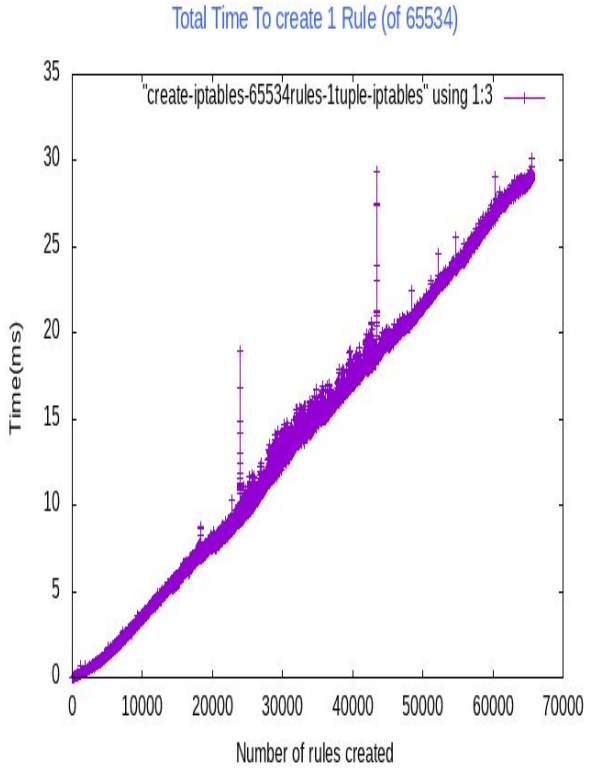


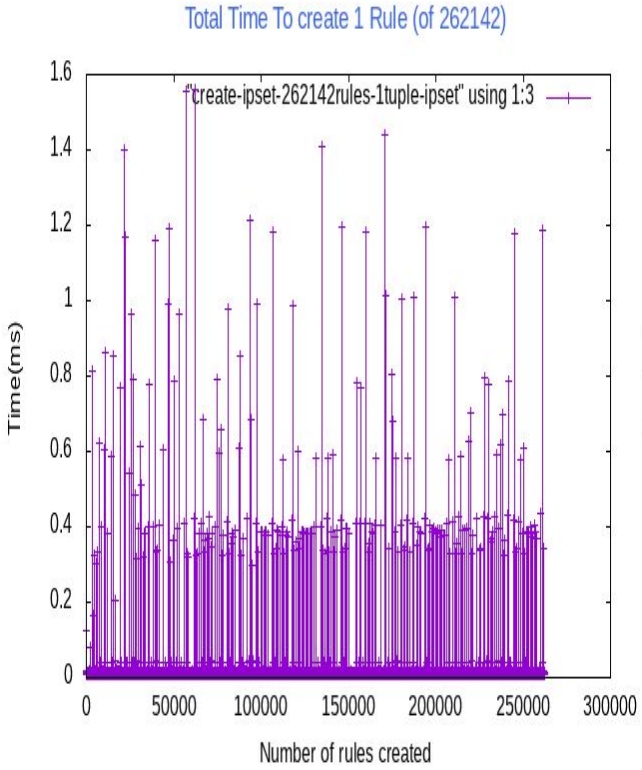
Host Egress TC NF OFF vs NF ON PPS/CPU

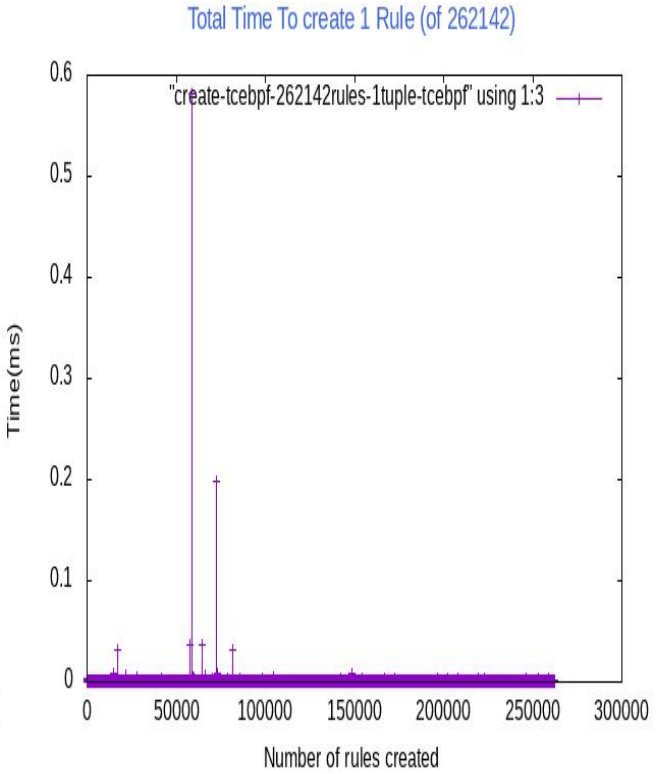


Controlpath Performance Comparison: The Trio vs PTables

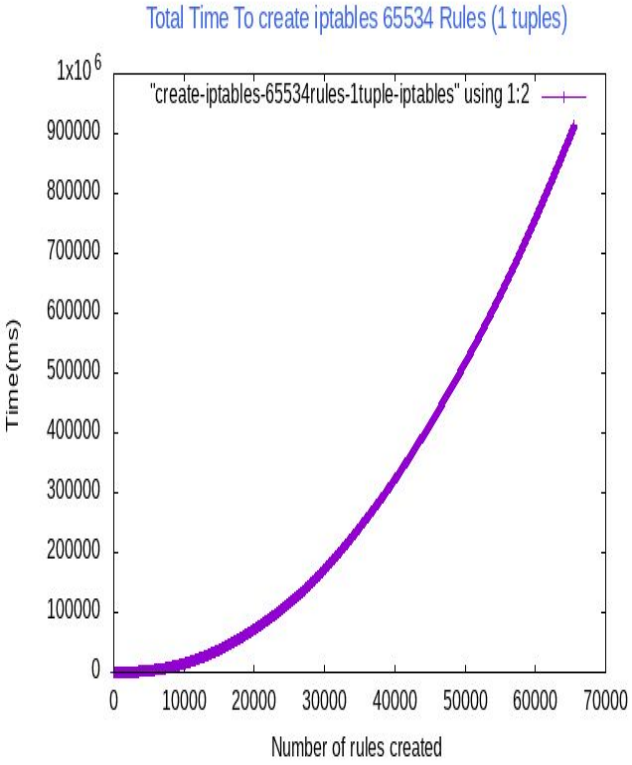
Control: Add Latency Nth+1 Rule

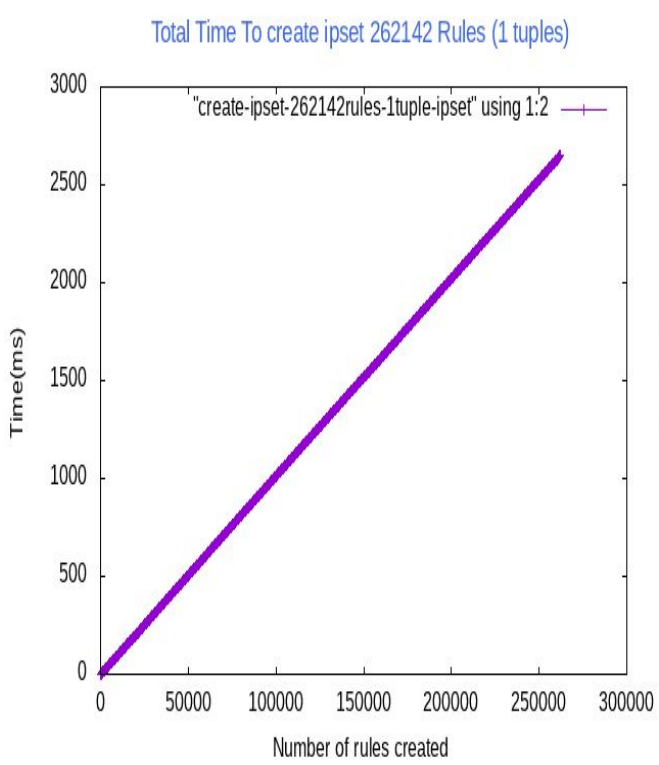


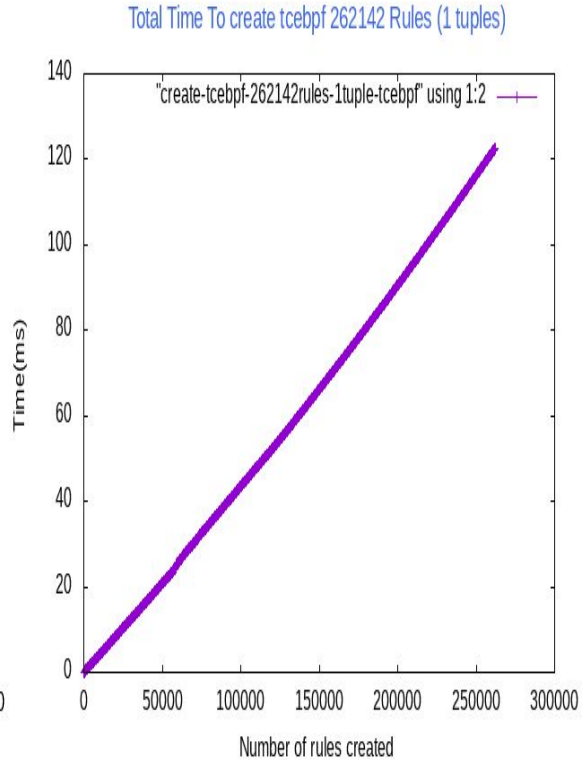




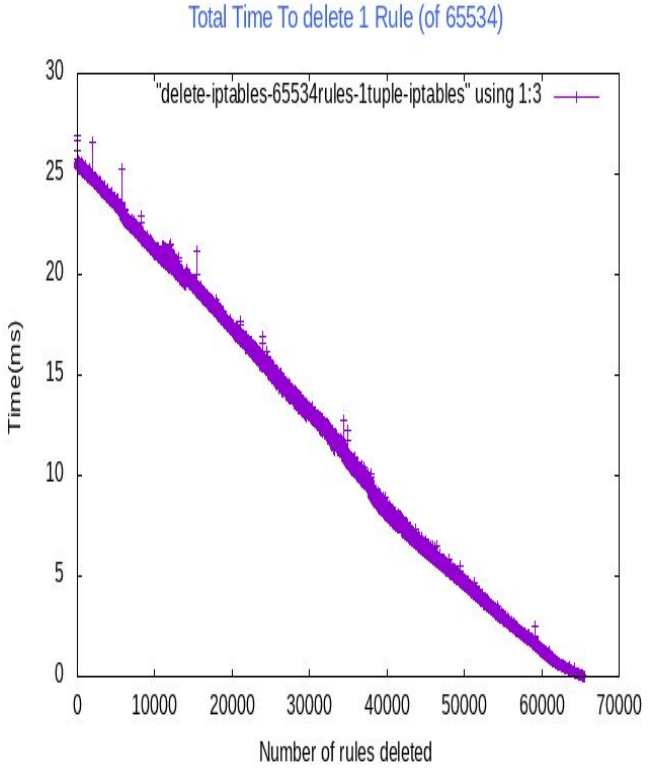
Control: Latency For Adding N Rules

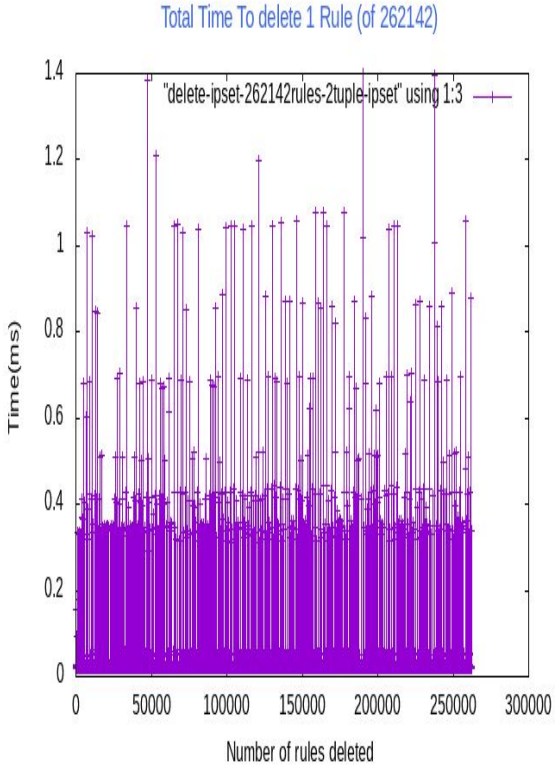


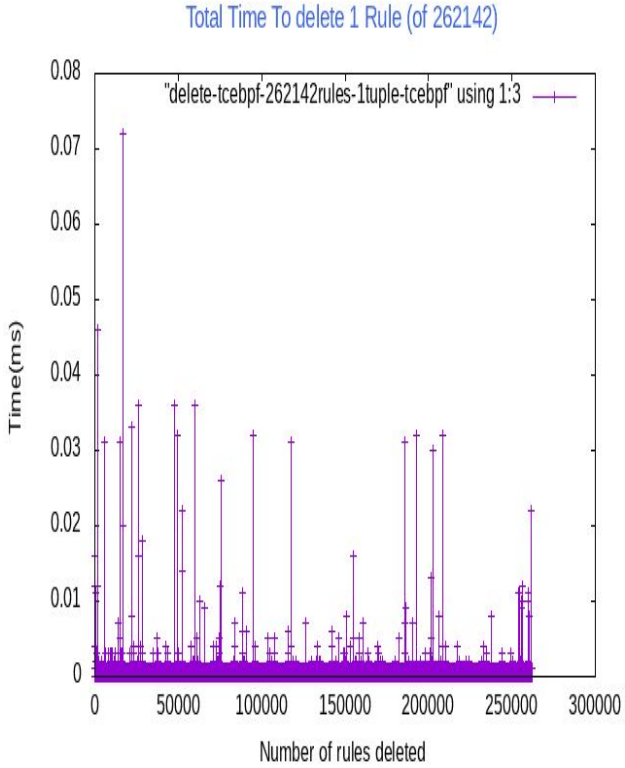




Control: Latency Deleting Nth Rule

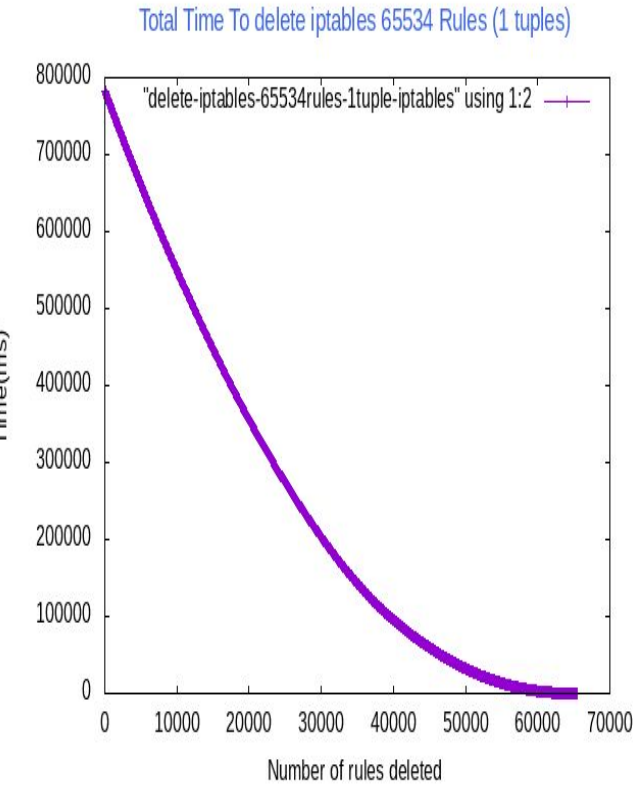


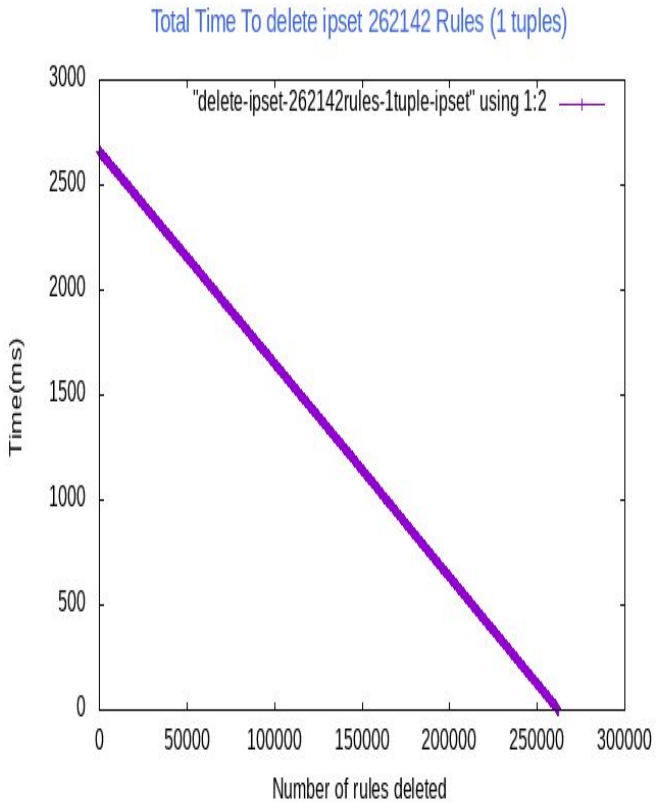


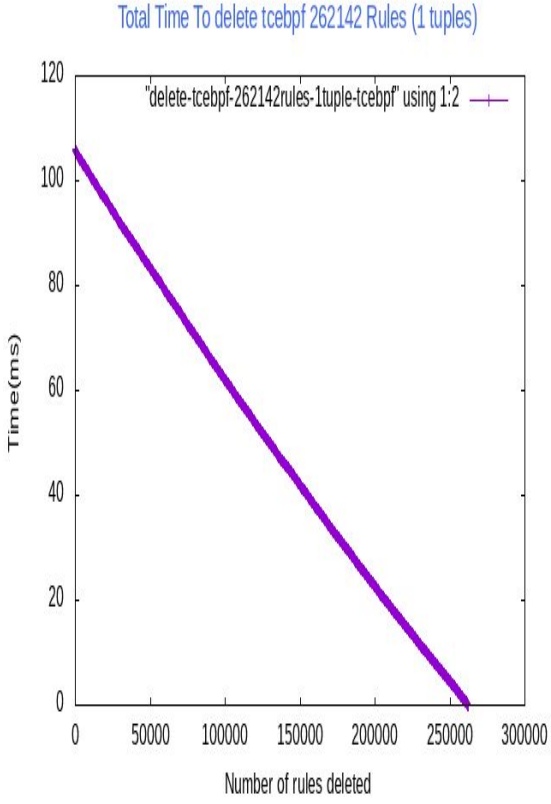


Control: Latency For Deleting N Rules

Incrementally 1 Rule at a time







Terminology: Rulesets vs Rules

- We call each iptables-equivalent entry a ruleset
  - priority in case of iptables is implicit in rule ordering
- RS:=<PRIO>[SRCIPGR][DSTIPGR]<PROTOINFO>, where:
  - PROTOINFO := <SPORTGR* DPORTGR* proto*>
  - SRCIPGR and DSTIPGR := PREFIX/MASK of V4/v6 (LPM)
- In Anglais, each ruleset has the following elements:
  - at most One srcIP CIDR table
  - at most one dstIP CIDR table
  - 1 or 0 Protocol selection
  - Source and/or destination port {ranges, lists}, missing means Wildcard
  - a group of 1 or more network interfaces
  - a direction (Ingress or Egress)